Binary Search:

Definition: A Binary Search is a Simp
Searching technique which can be appli
if the items to be compared are either
in ascending order or descending Order.

Analogy: The general idea used in binar
search for the telephone number of a pers
in the telephone-telephone directory. Obvio
we do not search for a name from the ver
first page. i,e we do not use linear Sear
instead, we open the book in the mid
and the name is compared with the name
the middle of the book. if the name found
the corresponding telephone number is
retrived and the search stops. Otherwis
we search either the left part of the book
or right-part-of the book. if the name to
searched is less than the middle element
search towards left side. Otherwise, searc
towards right-side. The procedure is repe
till name item is found or the name is not
found.

Design: Once we know that Concept of
Binary search, we can easily implement.

How to Search for key in a list of element

Initial: Find the initial values.

In the array a shown below, item 10 in position 0. in the first position and 90 in position 8 is the last element as shown below:

key
50

10	20	30	40	50	60	70	80	90
a[0]	a[1]	a[2]	a[3]	a[4]	a[5]	a[6]	a[7]	a[8]
↑								↑
Low								high

So, initial values are: low = 0 and high = 8
= 9-1
= n-1 (general)

So, in general initial values are

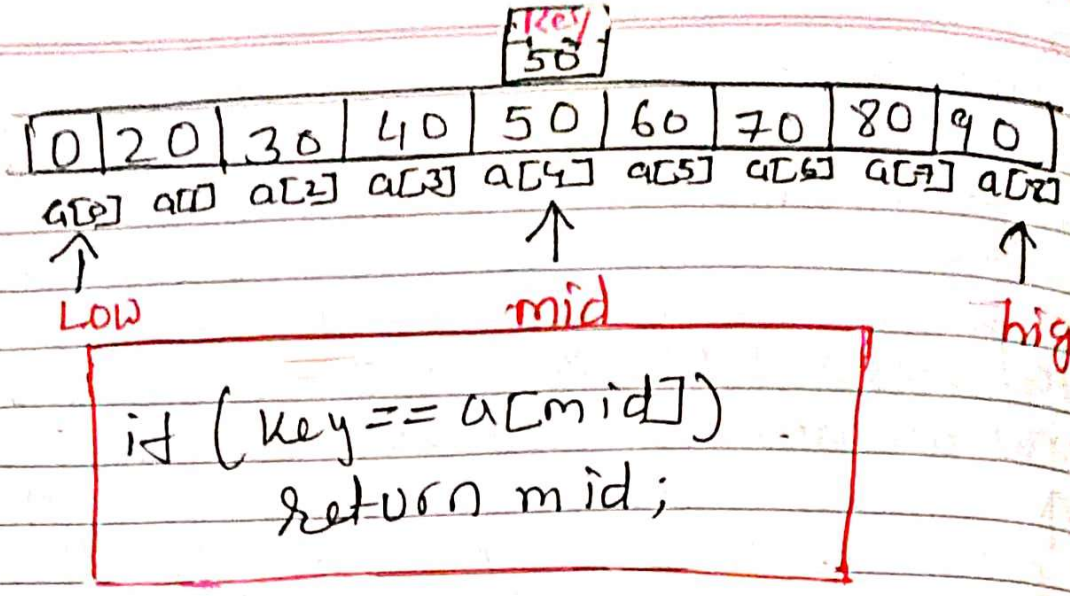
low = 0
high = n-1

Step1: Divide the array into two parts: if low is considered as the position of the first element and high as the position of the last element, the position of the middle element can be obtained using the statement.

mid ← (low+high)/2

Step2: Search key in the middle array:

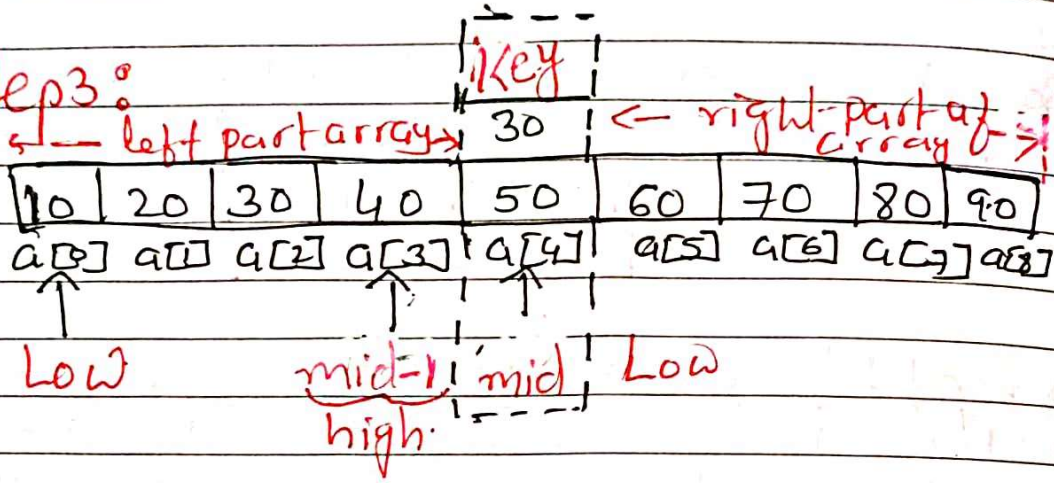
The key to be searched is compared with middle element, The pictorical representa-




```
if (key == a[mid])
    return mid;
```

if key is same as a[mid], then return the position mid indicating "Search is Successful".

Step3:

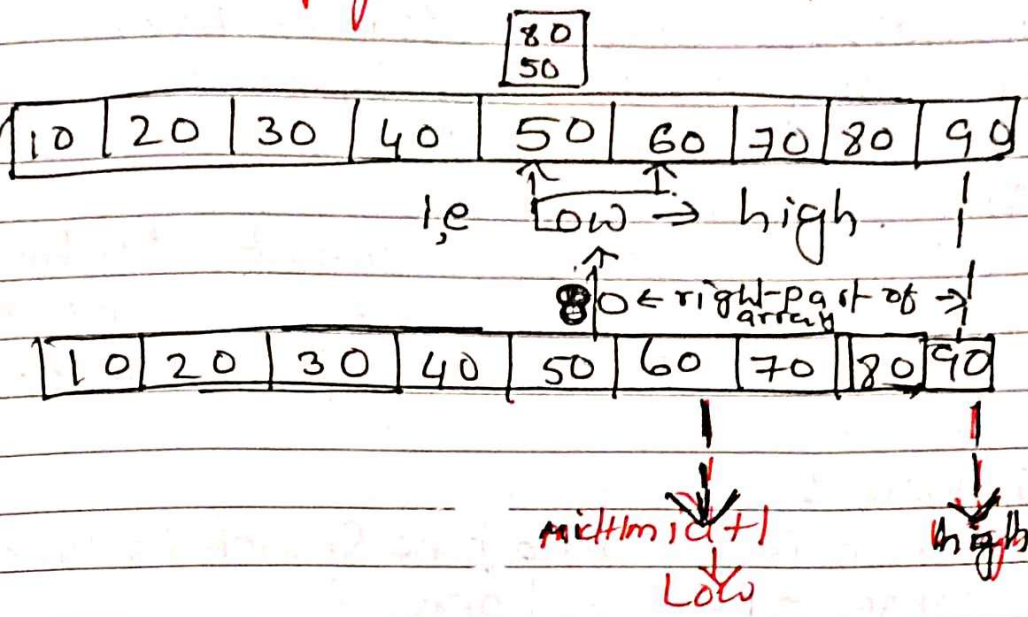


Search key in the left part of array:
if key is less than the middle element, the left part of list has to be Compared from Low to mid-1. As shown in diagram below:
i.e Low to high.

NOTE: Position of low has no been Changed But position of high is changed to mid-1
The equivalent code can be written as

```
if (key < a[mid])
```

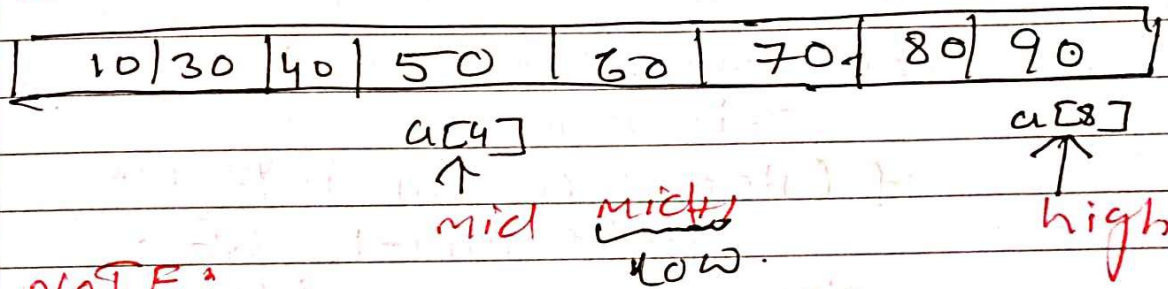
Step 4: Search the key is greater than middle element- right part of the array has to be considered from mid+1 to high as shown in figure below. i,e low to high.



The equivalent code can be written as if (key > a[mid]), low = mid+1.

```
if key = a[mid]
```

NOTE:



NOTE:

The equivalent code can be written as:

if (key > a[mid]) low = mid+1.

Key is not present in the array

* Note all the steps from 1 to 4 have to be repeatedly repeatedly executed as long

Design: The Recursive design of algorithm consists of identifying the base case and general case. The base case occurs whenever an item is not there in the array. Search is successful.

Base Case (Item not found):
We know that when low is greater than high no element are present in the list and hence key is not present. In this we return -1 indicating, item is not present in the list. The code can be written as shown below.

```
if(low > high) return -1
```

Base Case (Item found):
if the above condition fails, we have to compare with middle element. So, we have to find mid point as shown below

```
mid= (low+high)/2;
```

The key to be searched is compared with middle element. If key found, then search stop. This is achieved using the if statement.

```
/* Item found at mid position */
if (key == a[mid])
    return mid;
```

General Case:

If above condition fails and if key is less than the middle element, the elements in the left part of the table have to be compared recursively running from low to mid-1 using the following statement:

```
if( key<a[mid]) // search in the left part of the array
  return search(key, a, low, mid-1);
```

Otherwise, the right part of the array has to be compared from mid+1 to high. This can be achieved recursively using the following statement.

```
if (key>a[mid]) /* search in the right part
  return search(key, a, mid+1, high)
```

Algorithm for Binary Search.

Algorithm Binary-Search(key, a, low, high)

// purpose: Search for an item in the list 'a' by using binary search.

// input:
// key - the element to be searched.
// a - the list of element where the searching take place
// low - the position of the first element in the array.
// high - the position of the last element in the arrey.

// output: Position is returned if search is successful otherwise -1 is returned.

```
if( low > high) return -1 // key item not found
if(key==a[mid]) return mid; // item found at mid position
if(key<a[mid]) return search(key,a,low,mid-1); // search in the left part
return Search(key,a,mid+1, high); // Search in the right part
```

Algorithm that calls above Binary-Search

Step1: [Input the NUMBER of elements]
read n

Step2: [Input the array elements]
for i ← 0 to n-1 do
read a[i]
end for

Step3: [Input the element to be searched]
read key

Step4: [Search for key item in the list]
flag = Binary_Search(key, a, 0, n-1)
if (flag == -1)
print "Item Not found"
else print "Item found"
end if

Step5: [Terminate the Program]
exit

Analysis (Best Case):

The best case occurs when the ite
be searched is present in the middl
array. So, the total number of compar
required will be 1

So, the time Complexity of Binary-Searc
the best case is given by $\Omega(1)$

Analysis (Worst Case):

The worst case occurs when maximu
number of element comparisions are req
and the time complexity is given by
following recurrence relation.

$$T(n) = \begin{cases} 1 & \text{if } n = 1 \\ T(n/2) + 1 & \text{otherwise} \end{cases} \quad \text{---} (1)$$

$T(n/2)$ → Time required to Searche
the upper part or the lower part
of the array.

$+1$ → Time required to Compute
the middle element.

Now Consider the relation

$T(n) = T(n/2) + 1$

$= 1 + T(n/2)$ — (1)

$= 1 + [1 + T(\frac{n}{2^2})]$

$= 2 + T(n/2^2)$

$= 3 + T(\frac{n}{2^3})$

$= 4 + T(\frac{n}{2^4})$

$\vdots$

Finally $= i + T(\frac{n}{2^i})$ — (2)

$t(n) = 1 + t(n/2)$ — (4)

$t(n/2) = 1 + T(\frac{n}{2^2})$ replacing n by $n/2$ in eqn (1)

$n = 2^i$ — (3) take both side log of rhs we get

Substitute eqn (3) in eqn (2)

$\log_2 n = i \log_2 2$

$\log_2 n = i \cdot 1$

$i = \log_2 n$

$T(n) = i + T(\frac{n}{2^i})$

$= i + T(\frac{n}{n})$

$= i + T(1)$

$= i + 0$

$T(n) = \log_2 n$

So, Time complexity of Binary search.

$$T(n) = O(\log_2 n)$$